

Project 2: Regression and Symbolic Regression

Seth Cohen, Mahimna Patel, Krish Ranawat

October 6, 2026

Overview

This report extends the Project 1 exploratory analysis of three UCI datasets (Auto MPG, Concrete Compressive Strength, Airfoil Self-Noise) with regression and symbolic regression. Five techniques are evaluated using **ScalaTion**, Python's **Statsmodels**, and **PySR**, following the model allocation in the assignment:

1. **Regression** (ScalaTion + Statsmodels): all 3 datasets.
2. **Regularized Regression**, Ridge and Lasso (ScalaTion + Statsmodels): Auto MPG only.
3. **Forward Feature Selection** (ScalaTion): Auto MPG and Concrete.
4. **Transformed Regression**, log/sqrt/Box-Cox (ScalaTion): Auto MPG and Airfoil.
5. **Symbolic Regression** (ScalaTion + PySR): all 3 datasets.

Unless noted otherwise, models use every available predictor for a dataset (no manual feature pre-selection), so results here are not directly comparable to Project 1's two-feature simple regressions. Full console logs for every ScalaTion run are in `results/<dataset>/scalation_output.txt`; a compact digest of coefficients and QoF is in `results/<dataset>/scalation_digest.txt`.

Reproducing the results. The Python scripts in `python/` read `data/` relative to the repository and need `statsmodels`, and `pysr` for Section 5. To run the ScalaTion code, copy `scala/Project2.scala` into a ScalaTion app at `src/main/scala/project2/`, set the environment variable `PROJECT2_DATA_DIR` to this repository's `data/` folder, and run `sbt "runMain project2.project2_autoMPG"` (likewise `project2_concrete` and `project2_airfoil`).

1 Regression

1.1 Auto MPG

ScalaTion's `Regression` and Statsmodels' OLS were fit on `mpg` against all 7 predictors (`cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model_year`, `origin`). Both tools solve the same least-squares problem and, as expected, agree to at least 3 decimal places on every coefficient and QoF metric.

Table 1: Auto MPG multiple regression coefficients (ScalaTion; Statsmodels agrees).

Term	Coefficient
intercept	-17.218
cylinders	-0.4934
displacement	0.01990
horsepower	-0.01695
weight	-0.006474
acceleration	0.08058
model_year	0.7508
origin	1.4261

QoF: $R^2 = 0.8215$, adj. $R^2 = 0.8182$, RMSE = 3.294, $F(7,384) = 252.4$, $p < 0.001$. 5-fold cross-validation gives a very similar mean $R^2 = 0.809$ (fold range 0.79–0.84; see `scalation_output.txt`), so the model generalizes well and is not overfit.

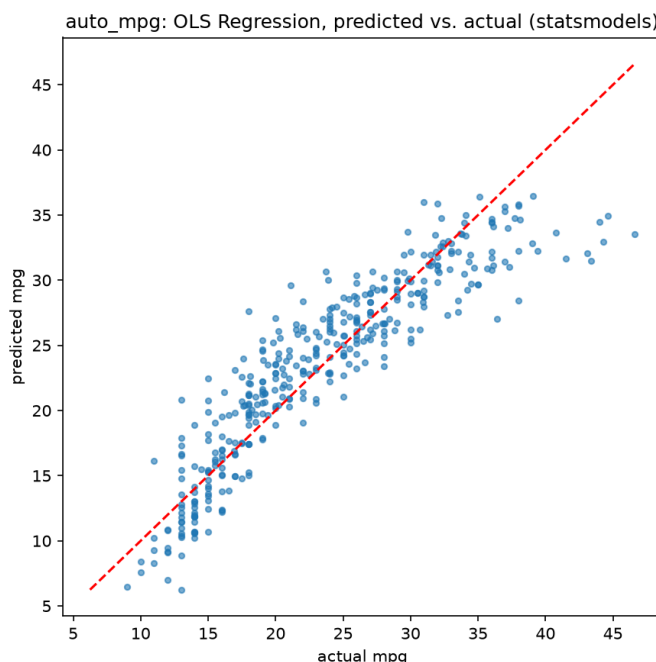


Figure 1: Auto MPG: Statsmodels OLS, predicted vs. actual mpg.

Interpretation: weight, model_year, and origin dominate: every extra 1000 lbs costs about

6.5 mpg, each model year (1970–1982) is worth about 0.75 mpg of efficiency gain, and, relative to US-built cars (origin 1), European cars (origin 2) are predicted about 1.4 mpg higher and Japanese cars (origin 3) about 2.9 mpg higher after controlling for size and era.¹ Displacement is the one predictor with a counter-intuitive sign: holding the other features fixed, its coefficient is positive (+0.0199 mpg per cubic inch) and significant ($p = 0.008$), even though on its own displacement is strongly negatively correlated with mpg. Cylinders’ coefficient has the expected negative sign but is not significant ($p = 0.13$). Both are symptoms of multicollinearity among the engine-size features (cylinders, displacement, horsepower and weight are all highly correlated with each other), so their individual coefficients are not reliable on their own.

1.2 Concrete Compressive Strength

Regression of `concrete_compressive_strength` on all 8 ingredient predictors.

Table 2: Concrete multiple regression coefficients (ScalaTion; Statsmodels agrees).

Term	Coefficient
intercept	−23.164
cement	0.1198
blast_furnace_slag	0.1038
fly_ash	0.0879
water	−0.1503
superplasticizer	0.2907
coarse_aggregate	0.0180
fine_aggregate	0.0202
age	0.1142

QoF: $R^2 = 0.6155$, adj. $R^2 = 0.6125$, RMSE = 10.35.

¹**origin** is categorical (1 = US, 2 = Europe, 3 = Japan) but, as in Project 1, it is entered as a single numeric column, which forces the origin 3 effect to be exactly twice the origin 2 effect. Dummy variables would remove that restriction.

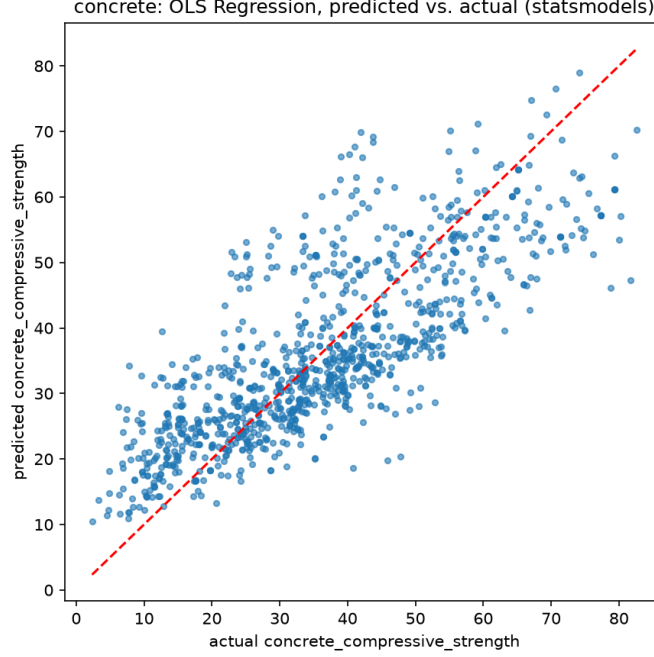


Figure 2: Concrete: Statsmodels OLS, predicted vs. actual compressive strength.

Interpretation: all three cementitious materials (cement, slag, fly ash) and superplasticizer raise strength, while water lowers it, matching the textbook water/cement-ratio relationship in concrete chemistry. Age has a positive effect (curing), and the two aggregate fractions matter least. Using all 8 predictors gives about 2.5 times Project 1’s best single-feature fit ($R^2 \approx 0.25$ for cement alone), confirming strength is genuinely multivariate, but 38% of the variance is still unexplained, likely due to nonlinear curing/age interactions (addressed in Section 5).

1.3 Airfoil Self-Noise

Regression of `scaled_sound_pressure_level` on all 5 predictors.

Table 3: Airfoil multiple regression coefficients (ScalaTion; Statsmodels agrees).

Term	Coefficient
intercept	132.83
frequency	−0.001282
angle_of_attack	−0.4219
chord_length	−35.69
free_stream_velocity	0.09985
suction_side_displacement_thickness	−147.30

QoF: $R^2 = 0.5157$, adj. $R^2 = 0.5141$, RMSE = 4.799.

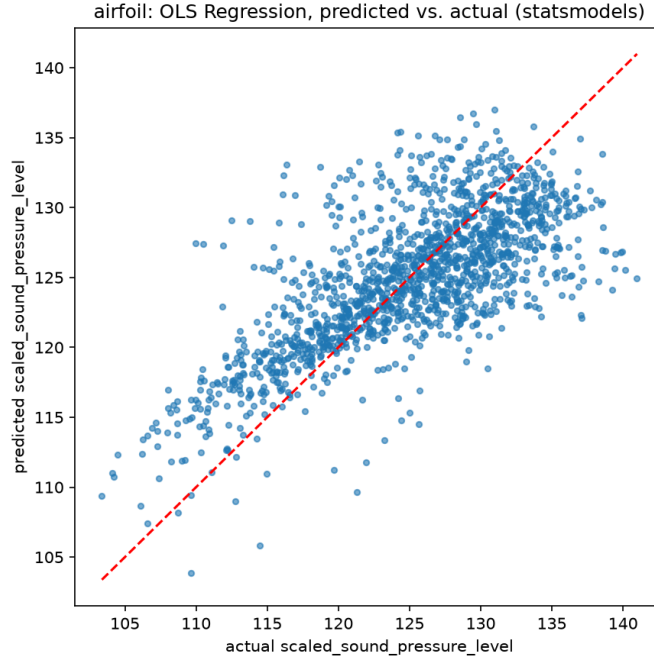


Figure 3: Airfoil: Statsmodels OLS, predicted vs. actual sound pressure level.

Interpretation: higher free-stream velocity is the only predictor that *raises* noise, matching aerodynamics (faster flow means louder self-noise); the other four all lower the fitted sound pressure level. Using all 5 predictors more than triples Project 1’s best single-feature fit ($R^2 \approx 0.15$ for frequency alone), but this remains the weakest linear fit of the three datasets, consistent with the Project 1 finding that boundary-layer self-noise is a genuinely nonlinear phenomenon.

2 Regularized Regression (Auto MPG only)

Ridge (L2) and Lasso (L1) were fit on Auto MPG in both tools. ScalaTion’s `RidgeRegression/LassoRegression` auto-center x and y (coefficients are on the original predictor scale, no intercept term); Statsmodels’ `OLS.fit_regularized` was run on standardized predictors with y centered (it has no intercept column, so in each cross-validation fold y is centered on the training-fold mean and that mean is added back to the predictions). For each of the four models, the penalty (λ for ScalaTion, α for Statsmodels) was chosen by 5-fold cross-validated SSE/MSE over a grid (see `results/auto_mpg/{ridge,lasso}_alpha_search.csv` and the `scaltion_output.txt` lambda-search log).

Table 4: Auto MPG Ridge and Lasso: chosen penalty, cross-validated error, and in-sample fit. ScalaTion’s CV error is the mean SSE over 5 test folds of 78 rows each (about 11.2 as an MSE); Statsmodels’ is the 5-fold mean MSE.

Tool	Model	Penalty (CV-chosen)	CV error	R^2
ScalaTion	Ridge	$\lambda = 10.0$	SSE 876.9	0.8214
ScalaTion	Lasso	$\lambda = 5.0$	SSE 877.0	0.8215
Statsmodels	Ridge	$\alpha = 0.01$	MSE 11.49	0.8208
Statsmodels	Lasso	$\alpha = 0.1$	MSE 11.46	0.8178
(unregularized OLS, Section 1)		–	–	0.8215

Table 5: Statsmodels standardized coefficients: Ridge vs. Lasso (Auto MPG).

Predictor	Ridge	Lasso
cylinders	−0.684	0.000
displacement	1.357	0.000
horsepower	−0.754	−0.381
weight	−4.910	−4.718
acceleration	0.105	0.000
model_year	2.700	2.646
origin	1.112	0.892

Interpretation: with only 7 predictors on 392 rows, unregularized OLS is not overfitting, so shrinkage barely moves R^2 (0.818–0.822 across all four models) or cross-validated error (both Statsmodels models are within 0.04 MSE of each other). Cross-validation picks a very light Ridge penalty ($\alpha = 0.01$), which leaves the OLS coefficients nearly intact, including the counter-intuitive positive `displacement` coefficient from Section 1. The value of regularization here is in *variable importance*, not accuracy: Lasso’s L1 penalty drives `cylinders`, `displacement`, and `acceleration` exactly to zero, leaving a sparse 4-predictor model (`horsepower`, `weight`, `model_year`, `origin`) with almost the same fit. This agrees with forward selection in Section 3, whose first three picks are `weight`, `model_year` and `origin`: those are the dominant drivers of fuel economy, and the engine-size predictors are largely redundant with each other once weight is in the model. ScalaTion’s Ridge/Lasso (which do not standardize predictors, so their λ values are on a different scale) told the same qualitative story: `acceleration`’s coefficient shrinks the most under Lasso (0.0806 $\rightarrow$ 0.0309), while `weight` and `model_year` are essentially unchanged.

3 Forward Feature Selection

ScalaTion’s `forwardSelAll` was run on Auto MPG and Concrete (Airfoil was left out, since it has only 5 predictors – too few for the ordering to be very informative – while the other two datasets have 7–8).

Selection criterion. At each step ScalaTion adds the predictor that most lowers sMAPE-IC, an information criterion equal to sMAPE plus a size penalty $2(k+1)/m$ (k predictors, m rows). It is computed on ScalaTion’s training split (the first 80% of rows), so it is *not* a cross-validated measure. For each step we also report $\bar{R}^2$ (training split) and the 5-fold cross-validated R_{cv}^2 . The three criteria do not always agree on the best subset size, so all three are shown; k counts predictors and excludes the intercept.

3.1 Auto MPG

Table 6: Auto MPG forward selection path. Best value of each criterion in bold.

k	Predictor added	sMAPE-IC ↓	$\bar{R}^2$ (%) ↑	R_{cv}^2 (%) ↑
1	weight	11.918	75.30	68.49
2	model_year	10.931	81.34	80.27
3	origin	10.603	81.70	81.00
4	cylinders	10.548	81.70	80.96
5	acceleration	10.485	81.70	80.75
6	displacement	10.500	81.72	80.88
7	horsepower	10.586	81.79	80.92

ScalaTion’s own criterion (sMAPE-IC) is lowest at $k = 5$, so forward selection keeps 5 of the 7 predictors and drops `displacement` and `horsepower`. $\bar{R}^2$ keeps creeping up to the full 7-predictor model, while cross-validated R_{cv}^2 peaks at only 3 predictors (`weight`, `model_year`, `origin`).

3.2 Concrete

Table 7: Concrete forward selection path. Best value of each criterion in bold.

k	Predictor added	sMAPE-IC ↓	$\bar{R}^2$ (%) ↑	R_{cv}^2 (%) ↑
1	superplasticizer	38.616	21.20	12.46
2	age	32.733	39.50	29.07
3	cement	29.722	51.18	47.11
4	blast_furnace_slag	28.447	55.80	54.16
5	fly_ash	27.056	59.15	56.40
6	water	26.739	61.45	60.22
7	fine_aggregate	26.772	61.51	60.18
8	coarse_aggregate	26.776	61.54	60.24

sMAPE-IC is lowest at $k = 6$, so forward selection keeps 6 of the 8 predictors and drops the two aggregate fractions. $\bar{R}^2$ and R_{cv}^2 are both highest for the full model, but only by 0.02–0.09 percentage points.

Interpretation: forward selection does shrink both models under ScalaTion’s criterion: AutoMPG from 7 to 5 predictors, and Concrete from 8 to 6. The predictors it drops add almost nothing. After AutoMPG’s third step and Concrete’s sixth, $\bar{R}^2$ and R_{cv}^2 move by less than 0.3 percentage points, so the choice between the reduced and full models is a near-tie that depends on the criterion. The more robust result is the *order* of selection, which acts as a model-based importance ranking. For AutoMPG, **weight**, **model_year** and **origin** come first, and cross-validation says those three alone predict as well as any larger subset. They are also three of the four predictors Lasso keeps in Section 2. The engine-size predictors (**displacement**, **horsepower**) enter last because they carry little information that **weight** doesn’t already. For Concrete, superplasticizer and age are selected before cement: once workability (superplasticizer) and curing time (age) are accounted for, cement adds less marginal predictive power than its Section 1 coefficient alone would suggest. The two aggregates, which also had the smallest Section 1 coefficients, are the ones dropped.

4 Transformed Regression

ScalaTion’s `TranRegression` was run on Auto MPG and Airfoil, applying `log(y)`, `sqrt(y)`, and `Box-Cox(y)` with $\lambda = 0.4$ (the library default; Yeo-Johnson is not exposed as a ready-made `Transform` in this ScalaTion build, so Box-Cox was used, as permitted by the rubric). Both target variables are strictly positive, so all three transforms are well-defined.

Table 8: Effect of transforming the response on in-sample R^2 .

Dataset	Untransformed	$\log(y)$	$\sqrt{y}$	Box-Cox(y), $\lambda = 0.4$
Auto MPG (mpg)	0.8215	0.8552	0.8437	0.8468
Airfoil (dB)	0.5157	0.5082	0.5122	0.5114

Interpretation: the two datasets tell opposite stories, which is itself the interesting finding. `mpg` is a reciprocal-like, right-skewed quantity (efficiency measured as distance/fuel rather than the more physically additive fuel/distance), so every transform toward a more-symmetric scale improves the fit; log is best, gaining $\Delta R^2 \approx 0.034$ over the untransformed model, consistent with standard practice of regressing `log(mpg)` or gallons-per-100-miles instead of `mpg` directly. `scaled_sound_pressure_level`, by contrast, is *already* a decibel (logarithmic) quantity, so applying a further log/sqrt/Box-Cox transform does not help: it slightly worsens R^2 , RMSE and sMAPE on the original scale, because it fights against a scale that is already close to linear in the underlying physics. This is a clean illustration that transformed regression should be guided by the target’s native scale, not applied by default.

5 Symbolic Regression

Both a power/cross-term expansion in ScalaTion and a genetic symbolic search in PySR were run on all three datasets.

5.1 Where the Two Approaches Fit

Symbolic regression covers several quite different search strategies. Table 9 summarizes the main families.

Table 9: Comparison of equation search and generation strategies in AI4Science.

Search Strategy	Mechanism	Discovery Speed	Complexity Limit	Best For
Pathwise Matrix (FFX)	Sparse deterministic linear regression over fixed libraries	Instantaneous (seconds)	Low to Medium	Ultra-fast baselines, large datasets
Time-Delay Search (NARX-SR)	Recurrent, lag-aware tree or orthogonal regressions	Fast to Medium (minutes)	Medium to High	Time series with feedback loops and historical memory
Physics Decomposition (AI Feynman)	Structural breakdown via physics priors (symmetries, units) + NN fit	Slow (hours)	High	Uncovering exact analytic laws of physics from raw data
Evolutionary Tree (PySR)	Genetic mutations and crossover of operator trees	Slow to Medium (mins/hrs)	High	Open-ended scientific discovery
Sparse Dynamics (SINDy)	Sequentially thresholded ridge regression	Fast	Medium	Physical systems (ODEs/PDEs)
Neural Generation (DSR)	Token-by-token sequence generation via RL	Medium	Medium	Finding specific structural patterns
Agentic Loop (SR-Scientist)	LLM code writing, execution, and optimization	Slow (agent loops)	Very High	Noisy data needing domain knowledge

The two tools used in this project come from different rows of this table. ScalaTion’s **SymbolicRegression** belongs to the first family: it builds a fixed library of candidate terms ($x^{0.5}$, x^2 , $x_i x_j$) and fits them with deterministic least squares. It differs from FFX in how it prunes the library: it uses greedy forward selection rather than a regularization path. PySR is the evolutionary-tree approach. It mutates and recombines expression trees, so it can produce forms the fixed library cannot represent, such as quotients and logarithms.

The table’s trade-offs show up directly in our results. The fixed-library fits ran in seconds (each dataset’s full ScalaTion run, including cross-validation and forward selection, took under 20 seconds) and were deterministic, but they can only reweight the terms they were given. PySR needed one to two minutes per dataset, plus a one-time Julia compilation, but it returned compact closed-form equations (Section 5.3) built from operators outside the library. None of our three

datasets is a time series or an ODE system, which is why the time-delay and sparse-dynamics families weren’t applicable here.

5.2 ScalaTion

Each dataset’s predictors were expanded with $x^{0.5}$, x^2 , an intercept, and all pairwise cross terms $x_i x_j$, then `forwardSelAll` was used to rank terms. As in Section 3, ScalaTion adds terms by sMAPE-IC on the training split, and we also track 5-fold cross-validated R_{cv}^2 . Term counts below exclude the intercept. An earlier attempt that also included x^{-1} and $x^{-0.5}$ terms produced a severely multicollinear, numerically unstable design matrix (coefficients up to 10^{15} , NaN standard errors, and an outright crash on Concrete) since a variable’s reciprocal and direct powers are highly correlated; the power set was narrowed to $\{0.5, 2\}$ to keep the expansion well-conditioned.

Table 10: ScalaTion symbolic regression. “Full” is the model with every candidate term (R^2 on all rows). The subsets are the forward-selection steps preferred by sMAPE-IC and by R_{cv}^2 . The last row gives Section 1’s linear regression for comparison.

	Auto MPG	Concrete	Airfoil
Candidate terms	35	44	20
R^2 , full model (all rows)	0.895	0.886	0.641
Best k by sMAPE-IC	27	37	16
Best k by R_{cv}^2	11	43	16
R_{cv}^2 at sMAPE-IC’s k	0.859	0.863	0.629
R_{cv}^2 at R_{cv}^2 ’s k	0.869	0.867	0.629
R_{cv}^2 , full model	0.860	0.866	0.628
R_{cv}^2 , linear regression (Section 1)	0.809	0.602	0.511

Interpretation: the power/cross-term expansion is a real improvement over linear regression on all three datasets, and it holds up out of sample: cross-validated R^2 rises from 0.81 to 0.86 (Auto MPG), 0.60 to 0.87 (Concrete), and 0.51 to 0.63 (Airfoil). Concrete gains the most, consistent with its strength depending on interactions between ingredients and age.

Forward selection behaves differently on each dataset. For Auto MPG, ScalaTion’s criterion keeps 27 of 35 terms, but cross-validation peaks much earlier: the first 11 terms ($R_{cv}^2 = 0.869$) predict slightly better than either the 27-term subset or the full model (0.860). The extra terms mostly fit noise, and a compact model is enough. For Concrete, sMAPE-IC stops at 37 of 44 terms, leaving out the last 7 added (superplasticizer_fly_ash, fly_ash_cement, coarse_aggregate^{0.5}, superplasticizer_cement, fine_aggregate², age_coarse_aggregate, age_cement). Cross-validation does not support dropping them: R_{cv}^2 is 0.863 at 37 terms and 0.866 for the full model. The criteria disagree here, and the honest conclusion is that those 7 terms neither clearly help nor clearly hurt. For Airfoil, all three criteria agree on the same 16 of 20 terms ($R_{cv}^2 = 0.629$, essentially equal to the full model’s 0.628). Even so, Airfoil remains the weakest of the three datasets, pointing to nonlinear structure beyond what a low-order power/cross-term basis captures.

5.3 PySR

PySR (binary operators $+$, $-$, $\times$, $\div$; unary operators square, sqrt, exp, log; 60 iterations, population size 33) searched directly over closed-form expressions rather than a fixed basis.

Table 11: PySR best equation per dataset (selected by PySR’s complexity/loss score).

Dataset	Equation	Complexity	R^2
Auto MPG	$\text{mpg} = \frac{(\text{model_year} - 45.714) \times 2104.74}{\text{weight}}$	7	0.8508
Concrete	$\text{strength} = \log(\text{fly_ash} + \text{water}) (\log(\text{age}) \text{cement} + 958.16) / \text{water} + \sqrt{\text{blast_furnace_slag} - 24.85}$	18	0.7564
Airfoil	$\text{spl} = 131.38 - \sqrt{25.816 \text{freq} \cdot \text{chord} \cdot \text{thickness}}$	10	0.5556

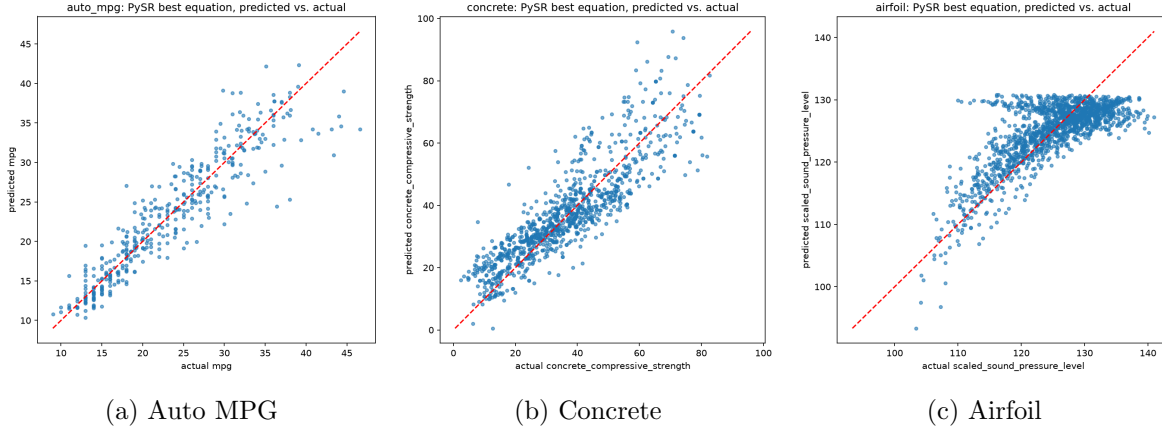


Figure 4: PySR best-equation predictions vs. actual, all three datasets.

Interpretation: all PySR R^2 values are in-sample on all rows, so they compare directly with the full-model R^2 in Table 10 and with Section 1. The two approaches trade accuracy for compactness differently. ScalaTion fits coefficients for a fixed basis of up to 44 terms. PySR searches over expression trees, so it can find operators the fixed basis doesn’t contain, such as division and logarithms.

PySR’s Auto MPG equation is close to physical reasoning: it finds that fuel economy scales with a (model-year-adjusted) constant divided by weight. It uses only 2 fitted constants, yet reaches $R^2 = 0.851$, *beating* the 8-parameter linear model ($R^2 = 0.822$), though not ScalaTion’s 35-term expansion ($R^2 = 0.895$). For Concrete, PySR clearly beats the linear model ($R^2 = 0.756$ vs. 0.616) but falls well short of ScalaTion’s 44-term expansion ($R^2 = 0.886$). What it offers instead is a compact, readable equation: a $\log(\text{age}) \times \text{cement}/\text{water}$ interaction that ScalaTion’s $\{x^{0.5}, x^2, x_i x_j\}$ basis cannot express. This lines up with concrete science, where strength gain follows a roughly logarithmic curing curve in age and depends on the cement/water ratio. For Airfoil, PySR’s equation ($R^2 = 0.556$) improves modestly on linear regression (0.516) but is below ScalaTion’s 20-term expansion ($R^2 = 0.641$). The **frequency** $\times$ **chord.length** $\times$ **thickness** product it finds is consistent with airfoil self-noise scaling, in which noise depends on these geometric and aerodynamic quantities jointly. The modest R^2 of every method in this section confirms that much of Airfoil’s noise variance is not captured by low-order combinations of these five predictors.

Cross-Section Summary

- **Linear regression accuracy ranks Auto MPG > Concrete > Airfoil**, reflecting how close to linear each underlying physical process is. Once interaction terms are allowed (Section 5), Concrete catches up with Auto MPG (cross-validated R^2 0.866 vs. 0.860), since much of its structure is in ingredient and age interactions. Airfoil stays last under every method.
- **Regularization** (Section 2) matters for interpretability (Lasso’s sparsity confirms which of Auto MPG’s correlated engine-size features are redundant) more than for raw accuracy, since the unregularized model was not overfitting to begin with.
- **Forward selection** (Section 3) shrank Auto MPG from 7 to 5 predictors and Concrete from 8 to 6 under ScalaTion’s sMAPE-IC criterion. The dropped predictors change $\bar{R}^2$ and R_{cv}^2 by under 0.3 percentage points. Its selection order agrees with Lasso: weight, model year and origin carry most of Auto MPG’s signal.
- **Transformed regression** (Section 4) only helps when the transform matches the target’s native scale: it clearly helped Auto MPG (already skewed) and did not help Airfoil (already log-scaled in dB).
- **Symbolic regression** (Section 5) gave the largest accuracy gains of any technique, and the gains hold up under cross-validation. ScalaTion’s power/cross-term expansion was the most accurate, while PySR’s search produced much more compact equations. Only PySR found terms outside a fixed polynomial basis: a 1/weight scaling for Auto MPG and a log(age) curing term for Concrete. Both have clear physical justification, not just statistical support.